

Working with files & data formats

COMPLETE LEARNING GUIDE

What You'll Learn

Real datasets rarely arrive as a single clean CSV — you'll pull data from exports that are CSV, JSON, Excel workbooks, or Parquet files, often with inconsistent encodings, mixed types, and sizes too large to comfortably fit in memory. This submodule goes deep on pandas' data-format tooling: how `pd.read_csv`, `pd.read_json`, `pd.read_excel`, and `pd.read_parquet` differ in what they expect and what they return, how to control dtype inference instead of trusting pandas' guesses, how to diagnose and fix encoding errors on real-world text exports, and how to process files too large to load in one shot using chunked reading. It closes with the mirror-image skill — writing cleaned data back out in the right format for the next step in a pipeline. This is the single most practically useful skill block for anyone whose day-to-day job involves 'get data out of whatever format it's in, cleanly, and hand it to the next stage.'

1. `pd.read_csv`: The Workhorse, and Its Hidden Complexity

`pd.read_csv()` looks like a one-liner but has dozens of parameters because real CSVs are inconsistent in dozens of ways. `dtype` lets you force a column's type instead of letting pandas guess (critical for things like zip codes or IDs that look numeric but should stay strings so leading zeros survive). `parse_dates` converts date-like columns to proper `datetime64` objects at load time instead of leaving them as strings you'd have to convert later. `na_values` lets you tell pandas which sentinel strings ('N/A', 'missing', '-', '999') should be treated as null, since every source system invents its own convention. `usecols` restricts which columns are actually loaded, which both speeds up reads and avoids wasting memory on columns you'll drop anyway.

PYTHON

```
import pandas as pd

df = pd.read_csv(
    "sales_export.csv",
    dtype={"customer_id": str, "zip_code": str},
    parse_dates=["order_date"],
    na_values=["N/A", "missing", "-"],
    usecols=["customer_id", "zip_code", "order_date", "amount"],
)

print(df.dtypes)
# customer_id          object
# zip_code             object
# order_date    datetime64[ns]
# amount              float64
```

2. JSON: Nested Structure and `json_normalize`

JSON data — especially from APIs — is frequently nested (a customer record with a list of orders, each with a list of line items), which doesn't map cleanly onto a flat table the way CSV does. `pd.read_json()` works well for already-flat JSON, but for nested JSON you'll load it with the standard `json` module first and then use `pd.json_normalize()` to flatten it into rows and columns, optionally exploding nested lists into their own rows. The `orient` parameter on `read_json/to_json` controls the on-disk shape ('records' — a list of row-dicts — is the most common and most portable).

PYTHON

```
import json
import pandas as pd

with open("customers.json") as f:
    raw = json.load(f)

# raw looks like: [{"id": 1, "name": "Aisha", "address": {"city": "Pune", "zip":
411001}}]
df = pd.json_normalize(raw, sep="_")
print(df.columns.tolist())
# ['id', 'name', 'address_city', 'address_zip']
```

3. Excel: Sheets, Engines, and Multi-Sheet Workbooks

`pd.read_excel()` reads `.xlsx/.xls` workbooks and requires an underlying engine library — `openpyxl` for modern `.xlsx` files, and (less commonly now) `xlrd` for legacy `.xls`. `sheet_name` selects which sheet to load: a name or index for a single sheet, a list for several (returned as a dict of DataFrames), or `None` to load every sheet in the workbook at once. Excel files frequently have a title row or merged header cells above the real data, which is what the `skiprows` and `header` parameters exist to handle — always inspect the first few rows before assuming `header=0` is correct.

- `pip install openpyxl` is required for `.xlsx` — pandas raises an `ImportError` with a clear message if it's missing, don't be surprised by it.
- `pd.read_excel('report.xlsx', sheet_name=None)` returns `{sheet_name: DataFrame}` for every sheet — convenient for workbooks with one sheet per region or month.
- Excel often stores numbers formatted as text (a common export quirk) — check dtypes after loading, the same way you would for CSV.

4. Parquet: The Columnar Format for Serious Data Volume

Parquet is a binary, columnar storage format (versus CSV/JSON's row-oriented, plain-text layout) built for analytics workloads. Because it stores each column contiguously and compressed, it is dramatically smaller on disk than the equivalent CSV, and reading it is faster because pandas/pyarrow can read only the columns you ask for without scanning the whole file — CSV always has to be read row by row from the top. Parquet also embeds its own schema (column names and types), so there's no dtype-guessing step the way there is with CSV: what you read back is exactly the type that was written. `pd.read_parquet()` and `df.to_parquet()` require the `pyarrow` (recommended) or `fastparquet` engine installed.

PYTHON

```
import pandas as pd

# Write once, compressed and schema-typed
df.to_parquet("sales.parquet", engine="pyarrow", compression="snappy",
index=False)

# Read back – only the columns you need, no dtype guessing
subset = pd.read_parquet("sales.parquet", columns=["customer_id", "amount"])
```

5. Diagnosing and Fixing Encoding Issues

A `UnicodeDecodeError` when reading a file almost always means the file wasn't written in the encoding you assumed (pandas defaults to UTF-8). Exports from older systems or Windows tools are frequently 'latin-1' (also called ISO-8859-1) or 'cp1252', which happily decode text that would raise an error under strict UTF-8. The pattern is: try the default first, and if it fails, re-attempt with `encoding='latin-1'` — it almost never raises an error itself (it maps every byte to a character), so the real risk is silently misreading characters rather than crashing, which is why validating a few known text fields after loading matters.

- `UnicodeDecodeError: 'utf-8' codec can't decode byte ...` is the single most common file-reading error you'll hit with real-world exports.
 - `encoding='latin-1'` (or 'cp1252' for Windows-originated files) is the standard fallback when UTF-8 fails.
 - The `chardet` or `charset-normalizer` libraries can auto-detect a file's encoding when you genuinely don't know it in advance.
 - Always re-check a few rows of text data after switching encodings — a successful read isn't proof the characters decoded correctly, only that nothing crashed.

6. Chunked Reading for Files Too Large for Memory

When a file is larger than available RAM, `pd.read_csv(..., chunksize=N)` returns an iterator of DataFrames of N rows each instead of loading everything at once, letting you process the file in fixed-memory pieces. This is the standard pattern for aggregating or filtering a huge file: iterate the chunks, apply the same operation to each, and combine the partial results (e.g. summing partial sums) rather than concatenating every chunk back into one giant DataFrame, which would defeat the purpose.

PYTHON

```
import pandas as pd

total_by_category = pd.Series(dtype="float64")

for chunk in pd.read_csv("huge_transactions.csv", chunksize=100_000):
    partial = chunk.groupby("category")["amount"].sum()
    total_by_category = total_by_category.add(partial, fill_value=0)

print(total_by_category.sort_values(ascending=False).head())
```

7. Writing Data Back Out Correctly

Every `to_*` method mirrors its `read_*` counterpart, but the most common mistake is on the write side: `df.to_csv('out.csv')` by default writes the DataFrame's index as an unnamed first column, which then shows up as a mysterious 'Unnamed: 0' column the next time the file is read — always pass `index=False` unless you specifically want the index preserved. When writing incrementally (e.g. appending daily batches to a running log file), `mode='a'` with `header=False` (after the first write) avoids repeating the header row on every append.

- `df.to_csv('out.csv', index=False)` — the `index=False` is easy to forget and creates a phantom column downstream.
- `df.to_parquet()` preserves dtypes exactly; `df.to_csv()` always writes plain text, so numeric precision and datetime formatting can subtly change round-trip.
- For appending to an existing file safely: `mode='a'`, `header=not file_exists` so the header is written exactly once.

Common Pitfalls

- Letting pandas guess dtypes on ID/zip-code columns, then losing leading zeros because they were silently cast to int.
- Forgetting `index=False` on `to_csv()`, producing an 'Unnamed: 0' column the next time the file is read back in.
- Loading an entire multi-gigabyte CSV into memory instead of using `chunksize`, causing `MemoryError` on modest machines.
- Assuming UTF-8 for every file and getting a confusing `UnicodeDecodeError` on exports from older or Windows-based systems.
- Using `pd.read_excel()` without checking `sheet_name` on a multi-sheet workbook and silently only getting the first sheet.
- Treating a successful read after switching encodings as proof of correctness, without spot-checking that text actually decoded right.
- Re-writing a huge dataset to CSV for a downstream pipeline step when Parquet would be smaller, faster, and preserve types.

Key Takeaways

- `dtype`, `parse_dates`, and `na_values` on `read_csv` turn 'pandas guessed wrong' into 'you told it exactly what to expect.'
- Nested JSON needs `pd.json_normalize()`, not `read_json()`, to flatten into a usable table.
- Parquet beats CSV/JSON for size, speed, and type-safety whenever the data is large or reused — reach for it by default in pipelines.
- An encoding error is a clue, not a dead end — `latin-1/cp1252` resolve the vast majority of real-world cases.
- `chunksize` turns 'this file is too big to load' into 'process it in fixed-size pieces' without buying more RAM.
- Always pass `index=False` on `to_csv()` unless you have a specific reason to keep the index column.

Learn More

pandas — IO Tools (Reading and Writing)

https://pandas.pydata.org/docs/user_guide/io.html

pandas — `pandas.read_csv` API Reference

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Apache Arrow — Parquet Format Overview

<https://arrow.apache.org/docs/python/parquet.html>